ICCS 102 – Coding Skills Assessment 1

Foundations: Variables, Strings, Numbers, Conditions, and Loops

Total: 30 points. Complete any 10 of the 15 regular exercises. You may attempt more; only your 10 highest exercise scores count. The bonus exercise is optional.

This assessment covers the following topics from class:

• variables,

• strings,

• numbers,

• arithmetic operations,

• type conversion,

• conditionals,

• `for` loops,

• `while` loops,

• and writing clean functions with type hints.

Submission Rules

• Create **one Python file per exercise**.

• Use the exact file names shown for each exercise.

• Put all exercise `.py` files in one folder, compress the folder into a ZIP file, and submit it as `[FirstName]_[FirstThreeLettersOfLastName].zip`. Example: `Barnabas_Smi.zip`. Do not include the square brackets.

• Each file must contain the required function named in that exercise.

• Write the exact function signature shown in the exercise.

• **IMPORTANT: Type hints are required.** This means your function parameters and return type must include types.

• Do **not** use `import` unless explicitly stated. The purpose of the assignment is for you to write your own functions.

• Do **not** use `input()` unless explicitly stated.

• Do **not** use `print()` inside the required functions.

• You may use `print()` outside the functions for testing. Remove or comment out test calls before submitting.

• Use Python 3.9 or later.

• Assume inputs have the stated types and meet the conditions in each exercise. You do not need to handle invalid inputs.

• Return results in the stated type and format. The examples are not the only inputs your functions should handle.

File Naming

Use the exact file names shown under each exercise.

Example:

```
f01_greeting_message.py
f02_full_name.py
```

What is a Function Signature?

A function signature is the first line that defines the function.

It includes:

• the function name,

• the parameters or inputs,

• the input types,

• the return type.

Example:

```
def greeting_message(name: str) -> str:
    ...
```

This means:

• the function name is `greeting_message`,

• it receives one input called `name`,

• the function expects `name` to be a `str`,

• the function returns a `str`.

Type hints describe expected types; Python does not automatically enforce them.

Types matter because they make your program clearer and help other programmers understand what kind of data your function expects.

To test your function, place this call below its definition:

```
print(greeting_message("Nick"))
```

Press Run in your IDE. The console should display `Hello, Nick!`.

Topic A — Variables, Strings, and Number Manipulation Exercises 1–5

Mini Lesson: Variables, Strings, and Numbers

A variable stores a value.

Examples:

```
name = "Alice"
age = 19
price = 45.5
```

Strings are used for text.

Numbers can be used in arithmetic expressions.

Example:

```
quantity = 3
total = price * quantity
```

Functions should receive input through parameters and return the result.

Mini Lesson: Types

Relevant types: `int`, `float`, `str`, `bool`, and `list`.

```
int    # Whole numbers such as -1, 0, 1, 2, 3, etc.
float  # Decimal numbers such as 1.1, 2.0, 3.73, etc.
str    # Text, such as "hello" or "Alice"
bool   # A logical value: True or False
list   # An ordered collection of values, such as [1, 2, 3]
```

1. Greeting Message

Problem: Return a greeting message in the exact format `Hello, <name>!`.

Function Name: `greeting_message`

Import: none

Function Signature:

```
def greeting_message(name: str) -> str:
    ...
```

Inputs

• `name: str`

Output

• `str`

Example inputs and their outputs

• `greeting_message("Alice")` → `"Hello, Alice!"`

• `greeting_message("Nick")` → `"Hello, Nick!"`

• `greeting_message("Somchai")` → `"Hello, Somchai!"`

Requirement

• Use string concatenation or an f-string.

• Return the final message.

File: `f01_greeting_message.py`

2. Full Name Formatter

Problem: Return a full name in the exact format `<first_name> <last_name>`. Capitalize the first character of each input and preserve all remaining characters. Assume both inputs are nonempty and have no leading or trailing spaces.

Function Name: `full_name`

Import: none

Function Signature:

```
def full_name(first_name: str, last_name: str) -> str:
    ...
```

Inputs

• `first_name: str`

• `last_name: str`

Output

• `str`

Example inputs and their outputs

• `full_name("Alice", "Wong")` → `"Alice Wong"`

• `full_name("Somchai", "jaidee")` → `"Somchai Jaidee"`

• `full_name("john", "Smith")` → `"John Smith"`

Requirement

• Combine the two strings with exactly one space between them.

• Convert the first character of each input to uppercase. Keep the remaining characters unchanged.

• Return the full name.

Hint

`text[0]` selects the first character, `text[1:]` selects the remaining characters, and `.upper()` converts a string to uppercase. Combine these operations for each input, then join the names with one space.

File: `f02_full_name.py`

3. Uppercase Message

Problem: Return the supplied message with all letters converted to uppercase. Keep spaces, numbers, and punctuation unchanged.

Function Name: `uppercase_message`

Import: none

Function Signature:

```
def uppercase_message(message: str) -> str:
    ...
```

Inputs

• `message: str`

Output

• `str`

Example inputs and their outputs

• `uppercase_message("Hello")` → `"HELLO"`

• `uppercase_message("Good morning!")` → `"GOOD MORNING!"`

• `uppercase_message("Python 123")` → `"PYTHON 123"`

• `uppercase_message("")` → `""`

Requirements

• Use the string method `.upper()`.

• Return the converted message.

Hint

Use `.upper()` to create an uppercase version of a string.

Example:

```
word = "goodbye"
capital_word = word.upper()
```

Here, `capital_word` contains `"GOODBYE"`. Spaces, numbers, and punctuation stay unchanged.

Use the same method on your function's parameter, then return the result.

File: `f03_uppercase_message.py`

4. Total Price

Problem: Calculate and return the total price using `price * quantity`.

Function Name: `total_price`

Import: none

Function Signature:

```
def total_price(price: float, quantity: int) -> float:
    ...
```

Inputs

• `price: float`

• `quantity: int`

Output

• `float`

Example inputs and their outputs

• `total_price(25.0, 3)` → `75.0`

• `total_price(49.5, 2)` → `99.0`

• `total_price(10.25, 4)` → `41.0`

Requirement

• Use multiplication.

• Return the total price as a number.

File: `f04_total_price.py`

5. First Three Characters

Problem: Return the first three characters of the supplied text. If the text has fewer than three characters, return the entire text. Keep the original capitalization.

Function Name: `first_three`

Import: none

Function Signature:

```
def first_three(text: str) -> str:
    ...
```

Inputs

• `text: str`

Output

• `str`

Example inputs and their outputs

• `first_three("Python")` → `"Pyt"`

• `first_three("Smith")` → `"Smi"`

• `first_three("Hi")` → `"Hi"`

• `first_three("")` → `""`

Requirements

• Use string slicing.

• Return the extracted text.

Hint

Use string slicing to take characters from the beginning of a string:

```
text[:number_of_characters]
```

Example:

```
word = "Monday"
beginning = word[:2]
```

Here, `beginning` contains `"Mo"`, the first two characters. Adjust the number for this exercise.

If the string is shorter than the requested slice, Python returns the available characters. An empty string stays empty, so you do not need an `if` statement.

File: `f05_first_three.py`

Topic B — Slightly More Advanced Exercises 6–10

Mini Lesson: Combining Operations and Conditions

Programs often need more than one idea at the same time.

For example:

• calculate a value,

• compare it with a condition,

• return a message depending on the result.

Example:

```
def is_adult(age: int) -> bool:
    return age >= 18
```

These exercises combine variables, arithmetic, strings, and conditionals.

6. Even or Odd

Problem: Return `"Even"` if the number is even, otherwise return `"Odd"`.

Function Name: `even_or_odd`

Import: none

Function Signature:

```
def even_or_odd(number: int) -> str:
    ...
```

Inputs

• `number: int`

Output

• `str`

Example inputs and their outputs

• `even_or_odd(4)` → `"Even"`

• `even_or_odd(7)` → `"Odd"`

• `even_or_odd(0)` → `"Even"`

• `even_or_odd(-3)` → `"Odd"`

Requirement

• Use the modulus operator `%`.

• Use an `if` statement.

File: `f06_even_or_odd.py`

7. Grade Category

Problem: Return a grade category based on the score. Assume the score is an integer from `0` to `100`.

Function Name: `grade_category`

Import: none

Function Signature:

```
def grade_category(score: int) -> str:
    ...
```

Inputs

• `score: int`

Output

• `str`

Grade rules

• `score >= 90` → `"A"`

• `score >= 80` and `< 90` → `"B"`

• `score >= 70` and `< 80` → `"C"`

• `score < 70` → `"Needs Improvement"`

Example inputs and their outputs

• `grade_category(95)` → `"A"`

• `grade_category(90)` → `"A"`

• `grade_category(82)` → `"B"`

• `grade_category(75)` → `"C"`

• `grade_category(60)` → `"Needs Improvement"`

Requirement

• Use `if`, `elif`, and `else`.

File: `f07_grade_category.py`

8. Simple Discount

Problem: Apply a 10% discount if the total price is at least 500 baht. Assume `total >= 0`.

Function Name: `discounted_price`

Import: none

Function Signature:

```
def discounted_price(total: float) -> float:
    ...
```

Inputs

• `total: float`

Output

• `float`

Discount rule

• If `total >= 500`, return `total * 0.9`.

• Otherwise, return `total` unchanged.

• No rounding is required.

Example inputs and their outputs

• `discounted_price(1000.0)` → `900.0`

• `discounted_price(500.0)` → `450.0`

• `discounted_price(300.0)` → `300.0`

• `discounted_price(750.5)` → `675.45`

Requirement

• Use an `if` statement.

• Use arithmetic operations.

File: `f08_discounted_price.py`

9. Password Length Check

Problem: Return `"Strong enough"` if the password has at least 8 characters. Otherwise, return `"Too short"`.

This exercise checks length only; it is not a complete check of password security.

Function Name: `password_strength`

Import: none

Function Signature:

```
def password_strength(password: str) -> str:
    ...
```

Inputs

• `password: str`

Output

• `str`

Example inputs and their outputs

• `password_strength("abcd")` → `"Too short"`

• `password_strength("password123")` → `"Strong enough"`

• `password_strength("12345678")` → `"Strong enough"`

• `password_strength("1234567")` → `"Too short"`

Requirement

• Use `len()`.

• Use an `if` statement.

File: `f09_password_strength.py`

10. Temperature Advice

Problem: Return advice based on the temperature in degrees Celsius.

Function Name: `temperature_advice`

Import: none

Function Signature:

```
def temperature_advice(temperature: float) -> str:
    ...
```

Inputs

• `temperature: float`

Output

• `str`

Advice rules

• `temperature >= 35` → `"Very hot, drink water"`

• `temperature >= 25` and `< 35` → `"Warm weather"`

• `temperature < 25` → `"Cool weather"`

Example inputs and their outputs

• `temperature_advice(36)` → `"Very hot, drink water"`

• `temperature_advice(35)` → `"Very hot, drink water"`

• `temperature_advice(30)` → `"Warm weather"`

• `temperature_advice(25)` → `"Warm weather"`

• `temperature_advice(20)` → `"Cool weather"`

Requirement

• Use `if`, `elif`, and `else`.

File: `f10_temperature_advice.py`

Topic C — For Loops and While Loops Exercises 11–15

Mini Lesson: Loops

A loop repeats code.

Use a `for` loop to go through a sequence, such as a range of numbers or the characters in a string.

Example:

```
for i in range(5):
    print(i)
```

Use a `while` loop when repetition depends on a condition.

Example:

```
count = 1
while count <= 5:
    count += 1
```

In functions, loops are often used to build totals, count values, or build lists.

To build a list, start with `values = []`. Use `values.append(value)` to add a value to its end.

11. Sum from 1 to N

Problem: Return the sum of numbers from 1 to n, including both 1 and n. Assume `n >= 1`.

Function Name: `sum_to_n`

Import: none

Function Signature:

```
def sum_to_n(n: int) -> int:
    ...
```

Inputs

• `n: int`

Output

• `int`

Example inputs and their outputs

• `sum_to_n(5)` → `15`

• `sum_to_n(10)` → `55`

• `sum_to_n(1)` → `1`

Requirement

• Use a `for` loop.

• Use a variable to store the running total.

• Do not use the built-in `sum()` function.

File: `f11_sum_to_n.py`

12. Count Down

Problem: Return a list of numbers counting down from n to 1, including both n and 1. Assume n >= 1.

Function Name: `count_down`

Import: none

Function Signature:

```
def count_down(n: int) -> list[int]:
    ...
```

Inputs

• `n: int`

Output

• `list[int]`

Example inputs and their outputs

• `count_down(5)` → `[5, 4, 3, 2, 1]`

• `count_down(3)` → `[3, 2, 1]`

• `count_down(1)` → `[1]`

Requirement

• Use a `while` loop.

• Return the list.

File: `f12_count_down.py`

13. Count Vowels

Problem: Count how many vowels are in a word. Count only a, e, i, o, and u. Do not count y as a vowel.

Function Name: `count_vowels`

Import: none

Function Signature:

```
def count_vowels(word: str) -> int:
    ...
```

Inputs

• `word: str`

Output

• `int`

Example inputs and their outputs

• `count_vowels("apple")` → `2`

• `count_vowels("banana")` → `3`

• `count_vowels("sky")` → `0`

• `count_vowels("AEIOU")` → `5`

Requirement

• Use a `for` loop.

• Treat uppercase and lowercase letters the same.

• Use `.lower()` to handle uppercase letters.

File: `f13_count_vowels.py`

14. First Multiple of Seven

Problem: Starting from `start`, return the first integer greater than or equal to `start` that is divisible by 7. Assume `start >= 1`.

Function Name: `first_multiple_of_seven`

Import: none

Function Signature:

```
def first_multiple_of_seven(start: int) -> int:
    ...
```

Inputs

• `start: int`

Output

• `int`

Example inputs and their outputs

• `first_multiple_of_seven(10)` → 14

• `first_multiple_of_seven(21)` → 21

• `first_multiple_of_seven(22)` → 28

• `first_multiple_of_seven(1)` → 7

Requirement

• Use a `while` loop.

• Use the modulus operator `%`.

File: `f14_first_multiple_of_seven.py`

15. Build Even Numbers List

Problem: Return a list of even numbers from 1 to n in ascending order. Include n if n is even. Assume `n >= 1`.

Function Name: `even_numbers_up_to`

Import: none

Function Signature:

```
def even_numbers_up_to(n: int) -> list[int]:
    ...
```

Inputs

• `n: int`

Output

• `list[int]`

Example inputs and their outputs

• `even_numbers_up_to(10)` → `[2, 4, 6, 8, 10]`

• `even_numbers_up_to(7)` → `[2, 4, 6]`

• `even_numbers_up_to(1)` → `[]`

• `even_numbers_up_to(2)` → `[2]`

Requirement

• Use a `for` loop.

• Use an `if` statement.

• Use `%` to check whether a number is even.

File: `f15_even_numbers_up_to.py`

Bonus Exercise — Hollow Star Box

Extra Credit: **2 points**

Problem: Return a text graphic that forms a hollow square box with n rows and n columns. Assume `n >= 2`.

The border of the box should use stars (*).

The inside of the box should use spaces.

Function Name: `hollow_star_box`

Import: none

Function Signature:

```
def hollow_star_box(n: int) -> str:
    ...
```

Inputs

• `n: int`

Output

• `str`

Important note about newline

• `\n` means **new line** in a string.

• It is used when one string should continue on the next line.

• For example, `"Hello\nWorld"` means:

```
Hello
World
```

The diagrams below show the returned strings as they appear when printed.

Example inputs and their outputs

• `hollow_star_box(3)` →

```
***
* *
***
```

• `hollow_star_box(5)` →

```
*****
*   *
*   *
*   *
*****
```

• `hollow_star_box(6)` →

```
******
*    *
*    *
*    *
*    *
******
```

Pattern rule

• The first row should contain only stars.

• The last row should contain only stars.

• In the middle rows, only the first and last positions should be stars.

• The inside of the box should be spaces.

Requirement

• Build the graphic step by step using loops.

• Do not use string repetition shortcuts such as `"*" * number` or `" " * number`.

• Return the full box as one string, with `\n` between rows and no newline after the last row.

• Do not print inside the function.

Testing note

To see the box clearly, you may test your function like this:

```
print(hollow_star_box(5))
```

But the required function itself should return the string.

File: `f_bonus_hollow_star_box.py`

Scoring and Deductions

Regular Exercises - 30 Points

Each regular exercise starts at **3 points** if its function returns the correct value, type, and format for every grading test. If any returned result is incorrect, the exercise receives **0 points**, with no partial credit for logic. Tests use only inputs permitted by the instructions.

For an otherwise correct solution, apply the following deductions separately to each affected exercise:

Issue	Deduction
Missing or incorrect parameter type hint	-0.25 for each affected parameter
Missing or incorrect return type annotation	-0.25 per exercise
Incorrect parameter name, where the function remains callable as specified	-0.25 for each affected parameter
Unmet explicit method requirement, such as a required loop, operator, or string method	-0.25 for each distinct requirement
Use of a prohibited input()	-0.25 for each occurrence
Uncommented print() statements, including test print() statements	-0.25 per statement

Count each print() statement only once, whether it appears inside a function or is used for testing. For example, `print(greeting_message("Nick"))` incurs a deduction; `# print(greeting_message("Nick"))` does not. Comments and commented-out test calls do not incur deductions. An exercise score cannot fall below zero.

Conditions Resulting in Zero

Issue	Consequence
Use of import or from ... import unless explicitly permitted in the exercise	0 for that exercise
Missing exercise file or incorrect Python filename	0 for that exercise
Missing required function or incorrect function name	0 for that exercise
Function cannot be called using the specified parameters	0 for that exercise
Syntax or other loading errors prevent the file from loading	0 for that exercise

No exercise in this assessment permits imports. Python filenames and function names must match the specification exactly, including capitalization. Do not apply further exercise-level deductions to an exercise already receiving zero.

Bonus and Final Score

The optional hollow star box is worth **2 points** if all grading tests pass. Otherwise, it receives **0 points**. The same annotation, method, and naming rules apply to the bonus, with its score never falling below zero.

Add the **10 highest regular exercise scores** after their deductions, add any bonus points, and cap this subtotal at **30 points**. Then apply the submission deduction below. The final score cannot be below zero.

Submission issue	Deduction
ZIP filename does not follow the required naming format	-3 points overall, applied once